Hash based structures

22B. Hash-Based Structures

Questions: 3 **Difficulty:** Junior to Senior

Q 22.8 Hash Maps / Objects — when do you use Map vs plain objects?

Difficulty: Junior **Asked at:** Every company

Both store key-value pairs. The differences matter when keys aren't strings or when performance matters.

JAVASCRIPT Copy

```
// Plain object — keys are always strings (or Symbols)
const userRoles = { admin: true, editor: true, viewer: false };
userRoles[123] = 'test';  // Key is silently converted to string "123"

// Map — keys can be ANY type (objects, functions, numbers)
const cache = new Map();
cache.set(userObject, { data: 'cached' });    // Object as key
cache.set(42, 'the answer');                   // Number as key (stays num
cache.set(document.body, { height: 1000 });    // DOM element as key
```

When to use Map over objects:

Feature	Object	Map
Key types	String, Symbol	Any type
Key order	Mostly ordered*	Insertion order guarantee
Size	Object.keys(). length (O(n))	map.size (O(1))
Iteration	for ... in (slow)	for ... of (fast, built-in)
Performance (many entries)	Slower for frequent add/ delete	Optimized for frequent add/delete operations
Prototype pollution	Possible	Not possible
JSON serialization	Native	Manual conversion needed

Frontend patterns with Map:

JAVASCRIPT Copy

```
// Component instance cache
const componentCache = new Map();
function getCachedComponent(config) {
  if (componentCache.has(config)) return componentCache.get(config);
  const component = createComponent(config);
  componentCache.set(config, component);
  return component;
}

// Frequency counting (common in interview problems)
function countWords(text) {
  const counts = new Map();
  for (const word of text.split(/\s+/)) {
    counts.set(word, (counts.get(word) || 0) + 1);
  }
  return counts;
}

// WeakMap - garbage-collectible keys (for DOM element metadata)
const elementData = new WeakMap();
function trackElement(element, data) {
  elementData.set(element, data);
  // When element is removed from DOM and no other references exist,
```

Q 22.9 Sets — when are they the right choice?

Difficulty: Junior **Asked at:** Amazon, Flipkart, Atlassian

Set stores unique values. O(1) for add, delete, and has (membership check). Use it whenever you need to check "is this item in the collection?" frequently.

JAVASCRIPT Copy

```
const seen = new Set();
seen.add('apple');
seen.add('banana');
seen.add('apple');    // Duplicate — ignored
seen.size;            // 2
seen.has('apple');    // true (O(1) — much faster than array.includes whic

// Deduplication — the most common use
const unique = [...new Set(arrayWithDuplicates)];

// Set operations
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5, 6]);

const union = new Set([...a, ...b]);                    // {1,2,3,4,5,6}
const intersection = new Set([...a].filter(x => b.has(x)));  // {3,4}
const difference = new Set([...a].filter(x => !b.has(x)));   // {1,2}
```

Frontend use cases:

- Track which items the user has seen (infinite scroll — don't re-render already-viewed items)
- Track selected items in a multi-select without duplicates
- Dependency resolution — track visited nodes to avoid cycles

- Feature flags — `new Set(['dark-mode', 'new-checkout', 'ai-search'])`
- Real-time user list — `new Set()` of connected user IDs (no duplicates when user reconnects)

Q 22.10 Bloom Filters — what are they and when would a frontend use one?

Difficulty: Senior **Asked at:** Google, Amazon

A Bloom filter is a space-efficient probabilistic data structure that answers "is this element in the set?" It can say "definitely no" or "probably yes" — never "definitely yes." False positives are possible, false negatives are not.

JAVASCRIPT Copy

```
class BloomFilter {
  constructor(size = 1000) {
    this.bits = new Uint8Array(size);
    this.size = size;
  }

  #hash1(str) {
    let hash = 0;
    for (const ch of str) hash = ((hash << 5) - hash + ch.charCodeAt(0)) |
    return Math.abs(hash) % this.size;
  }

  #hash2(str) {
    let hash = 5381;
    for (const ch of str) hash = ((hash << 5) + hash + ch.charCodeAt(0)) |
    return Math.abs(hash) % this.size;
  }

  add(item) {
    this.bits[this.#hash1(item)] = 1;
    this.bits[this.#hash2(item)] = 1;
  }

  mightContain(item) {
```

```
}

const filter = new BloomFilter(10000);
filter.add('hello');
filter.mightContain('hello');  // true (definitely added)
filter.mightContain('world');  // false (definitely NOT added)
filter.mightContain('help');   // possibly true (false positive - hash col
```

Frontend use cases:

- **Spell checker:** Check if a word might be in the dictionary (ship a 100KB Bloom filter instead of a 10MB word list). If the Bloom filter says "no," the word is definitely misspelled. If "yes," do a full server-side check.
- **URL deduplication:** In a web crawler or link checker, track visited URLs. A Set of 10 million URLs uses gigabytes. A Bloom filter uses megabytes with <1% false positive rate.
- **Content recommendation:** "Has the user already seen this article?" Check the Bloom filter client-side before making an API call.

In interviews, you're rarely asked to implement a Bloom filter. But knowing it exists and when it's useful (membership testing on large sets where space matters and false positives are tolerable) shows depth.

QUICK CHECK — no marks, just for you.

1. What does this snippet log?

JAVASCRIPT Copy

```
const obj = {};
obj[123] = 'a';
console.log(Object.keys(obj)[0] === '123');

const m = new Map();
```

```
m.set(42, 'the answer');
console.log(m.size);
```

- `true`
  `the answer`
  `1`
- `false`
  `undefined`
  `1`
- `true`
  `undefined`
  `1`
- `true`
  `undefined`
  `2`

Check answer

2. You repeatedly need to answer "is this item in the collection?" over a large collection. According to the lesson, why is a Set a better choice than an array?

- Arrays cannot hold unique values, so duplicates would break the membership check.
- Set.has is an O(1) membership check, while array.includes has to scan the array in O(n).
- A Set keeps its values sorted, which lets membership checks use binary search.
- Set.has compares values deeply, while array.includes only compares object references.

Check answer

3. A spell checker ships a Bloom filter of dictionary words to the browser. For a typed word, mightContain(word) returns false. What can you conclude?

The word is definitely not in the dictionary, so it is misspelled.

- The word is in the dictionary, but a hash collision caused the filter to miss it.
- The word is probably not in the dictionary, but a server check is still needed to be sure.
- The result is inconclusive, because a Bloom filter can report false negatives.

Check answer

Previous
Linear data structures

Next
Tree and graph structures

LEARN MORE

ARTICLES & DOCS

Hash table (VisuAlgo)
visualgo.net

VIDEOS



Hash table — illustrated
YouTube



Learn hash tables in 13 minutes
YouTube



Intro to hash tables & dictionaries
YouTube

Hand-picked references for this lesson.